

kdb 技术方案

业务需求

面向 Kubernetes 场景，为数据平台提供一套标准化、可声明式的数据库交付与运维能力，覆盖 MySQL、PostgreSQL 等常用引擎在 K8s 内的创建、扩缩容、关停保留、（规划中的）切换、备份与可观测能力。

核心交付形态为 `CRD + Operator`：用户/运维平台通过 CR（`KDBCluster` / `KDBInstance`）声明期望状态，`kdb-operator` 负责资源编排与状态收敛；实例运行时动作由同 Pod 生命周期的 Sidecar（`kdblet`）承载，避免控制器直接执行容器内命令。

★ **价值主张：**把数据库交付从“脚本化、人工化”提升为“平台化、声明式”，在统一模型下复用编排能力，降低交付与运维成本，并为后续自动化治理与演进（MGR/PXC 等）提供基础。

现状与目标

2.1 现状梳理

当前在 Kubernetes 内交付数据库通常存在以下问题：配置与部署方式分散（Helm/脚本/手工 YAML 混用），实例生命周期动作（初始化、探活、复制搭建、备份）强依赖人工操作或外部脚本，平台难以统一治理与审计；同时缺乏统一的观测与事件闭环，出现异常时定位成本高。

此外，不同数据库引擎、不同架构（单机、主从、主备）在运维动作与依赖上差异明显，导致“每一套引擎/架构都要重写一遍交付逻辑”，难以沉淀可复用能力。

2.2 业务目标

目标：

- **声明式交付：**通过 `KDBCluster` / `KDBInstance` 描述期望状态，Operator 通过 Reconcile 收敛到最终一致。
- **可插拔引擎：**MySQL、PostgreSQL 等引擎复用通用编排能力，引擎差异通过 StepManager/插件方式扩展。
- **分层解耦：**控制面只做资源编排与策略收敛；容器内动作由 `kdblet` 执行，控制器不直接进入容器。
- **可观测闭环：**指标 + 事件 + 日志三位一体，形成交付/运维的可追溯链路。

- 非目标（当前阶段）：
- 不承诺提供完整的多 AZ 共识或强一致方案（如 MGR/PXC），作为演进方向。
 - 不将“业务 SQL 级别运维”强耦合进 Operator；更多细粒度动作沉到 Sidecar 或外部运维平台。

方案设计

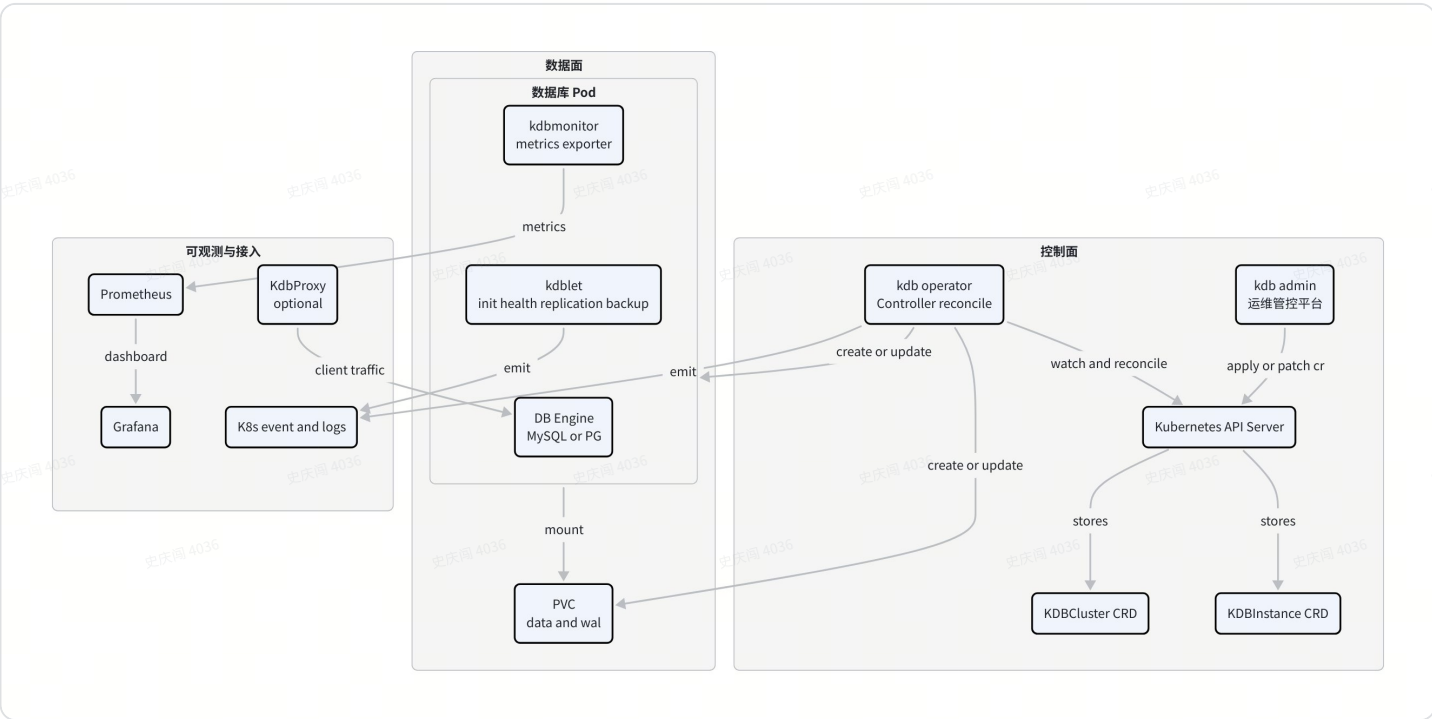
3.1 方案描述

整体采用“控制面 + 数据面 + 可观测/接入”的分层架构：

- 控制面 (kdb-operator)**：监听 CR 变更，编排 K8s 资源（StatefulSet、PVC、Service、ConfigMap、Secret、RBAC 等），并以 Step/Task 的方式组织 Reconcile，保证幂等与最终一致。
- 数据面 (DB Pod)**：数据库主容器 + `kdblet` + `kdbmonitor` 共 Pod 部署，`kdblet` 提供初始化、健康检查、复制搭建/切换、备份等动作接口，`kdbmonitor` 负责指标采集。
- 接入层 (可选)**：通过 `KdbProxy`（如 `proxysql/pgbouncer`）提供稳定连接、读写分离或连接池能力。

 **设计原则：** Operator 只做“编排与收敛”，Sidecar 负责“动作执行”。这使得控制器在权限与稳定性上更可控，也便于在不同引擎上复用同一套生命周期编排框架。

3.2 模块图 & 模块介绍

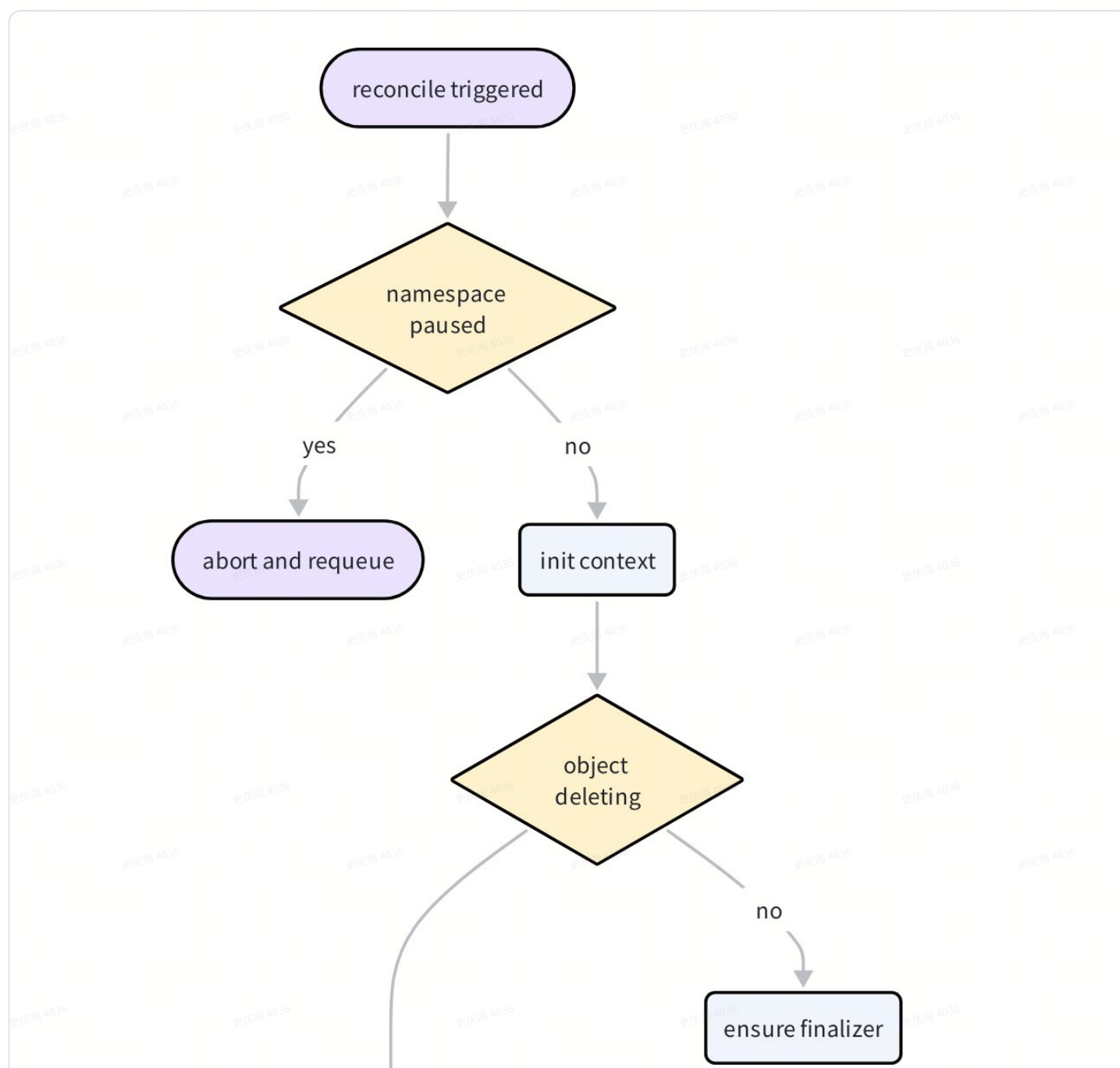


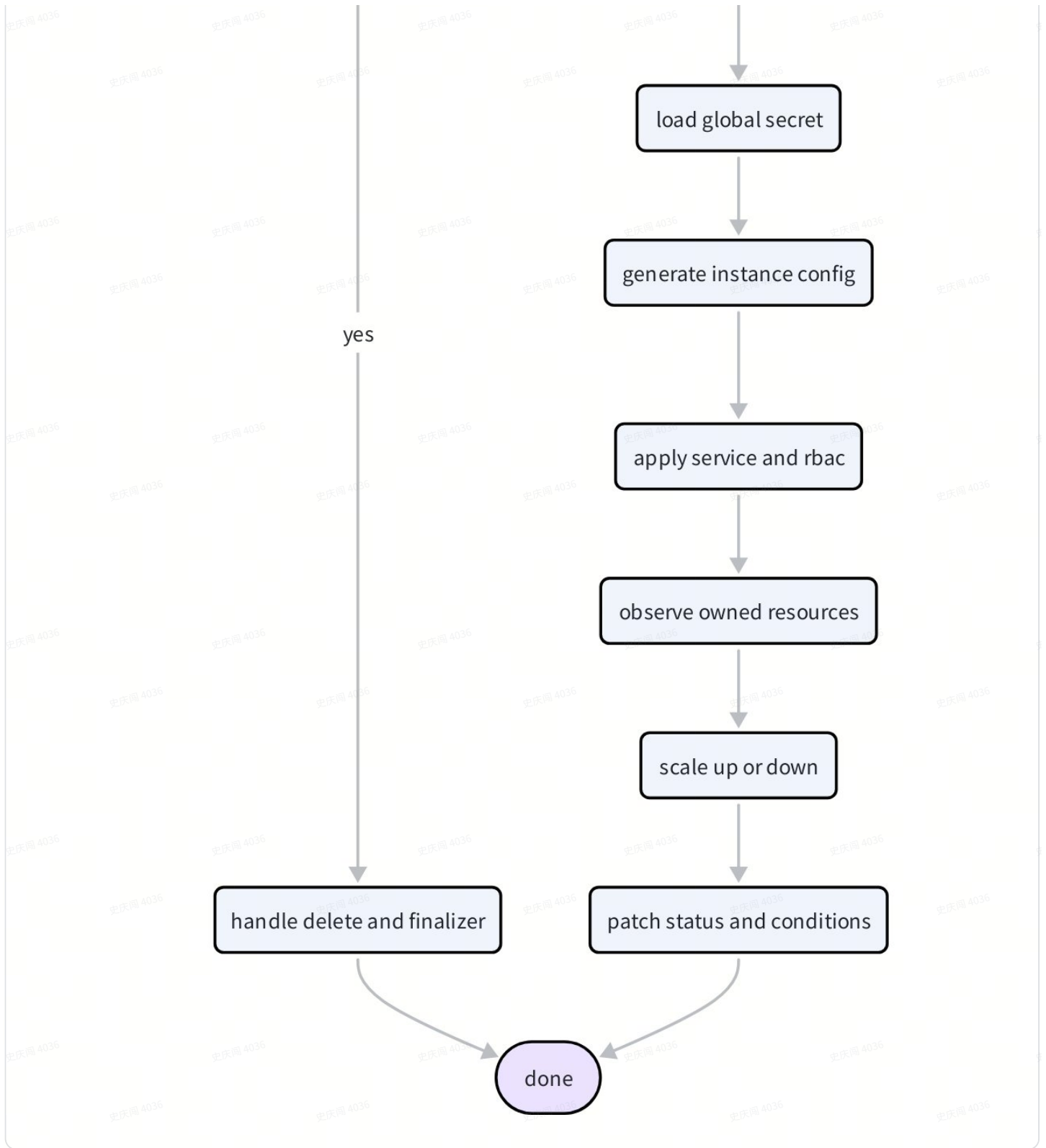
模块说明：

- **KDBCluster**：集群级资源，用于聚合多个 **KDBInstance**，描述一个集群由哪些实例构成以及采用何种部署架构。
- **KDBInstance**：实例级资源，通常映射为“一个 StatefulSet + PVC + Service + 配置与权限资源”的集合。
- **kdb-operator**：实现两个 Controller（Cluster/Instance），Cluster Controller 负责实例集合编排，Instance Controller 负责实例资源收敛与状态回写。
- **kdblet**：与数据库容器同生命周期，承载容器内动作的执行面（初始化/探活/复制/切换/备份）。
- **kdbmonitor**：Exporter/采集器，输出指标供 Prometheus 抓取。

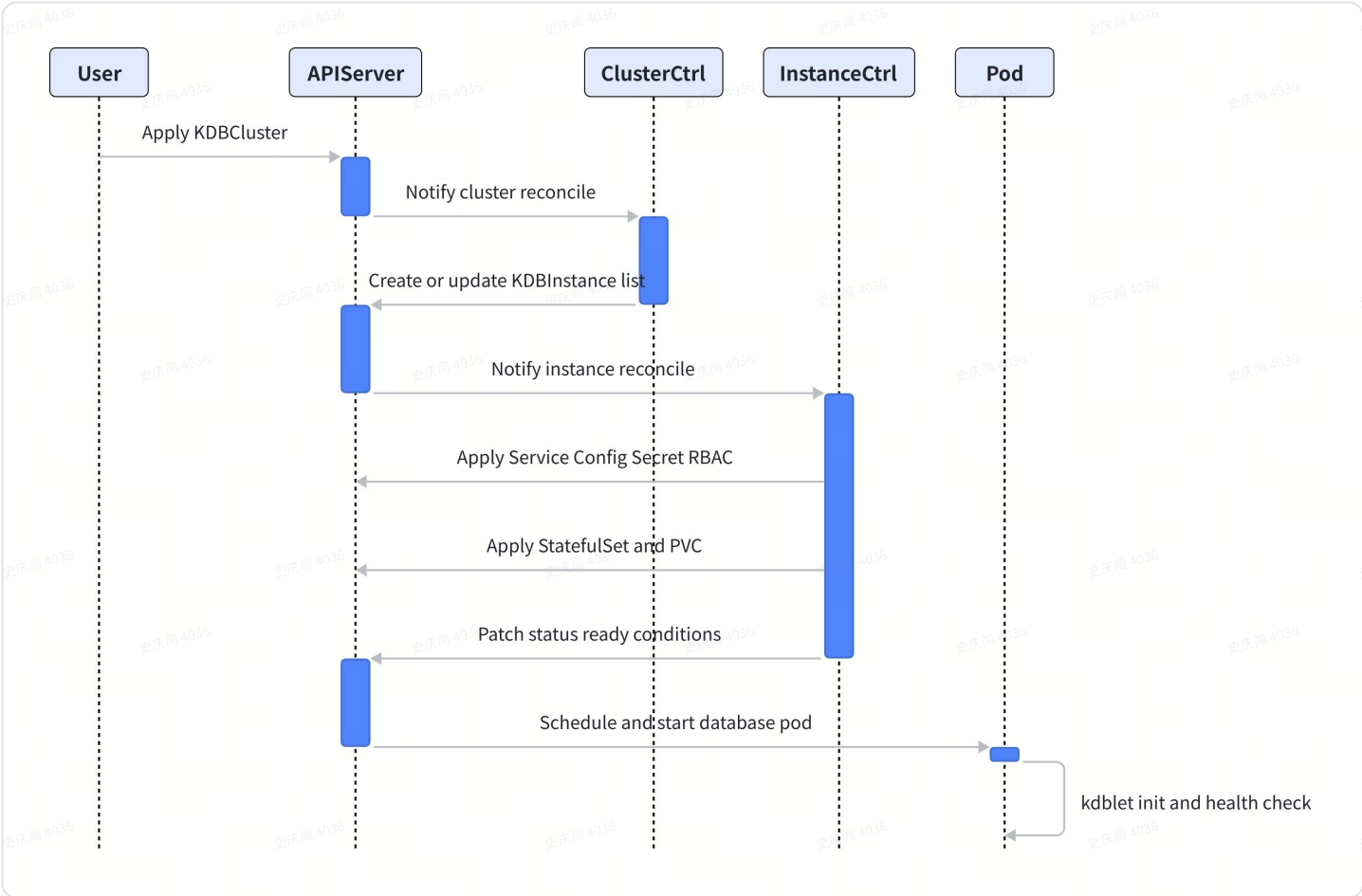
3.3 时序图、流程图 & 流程说明

Reconcile 收敛流程（Instance Controller）

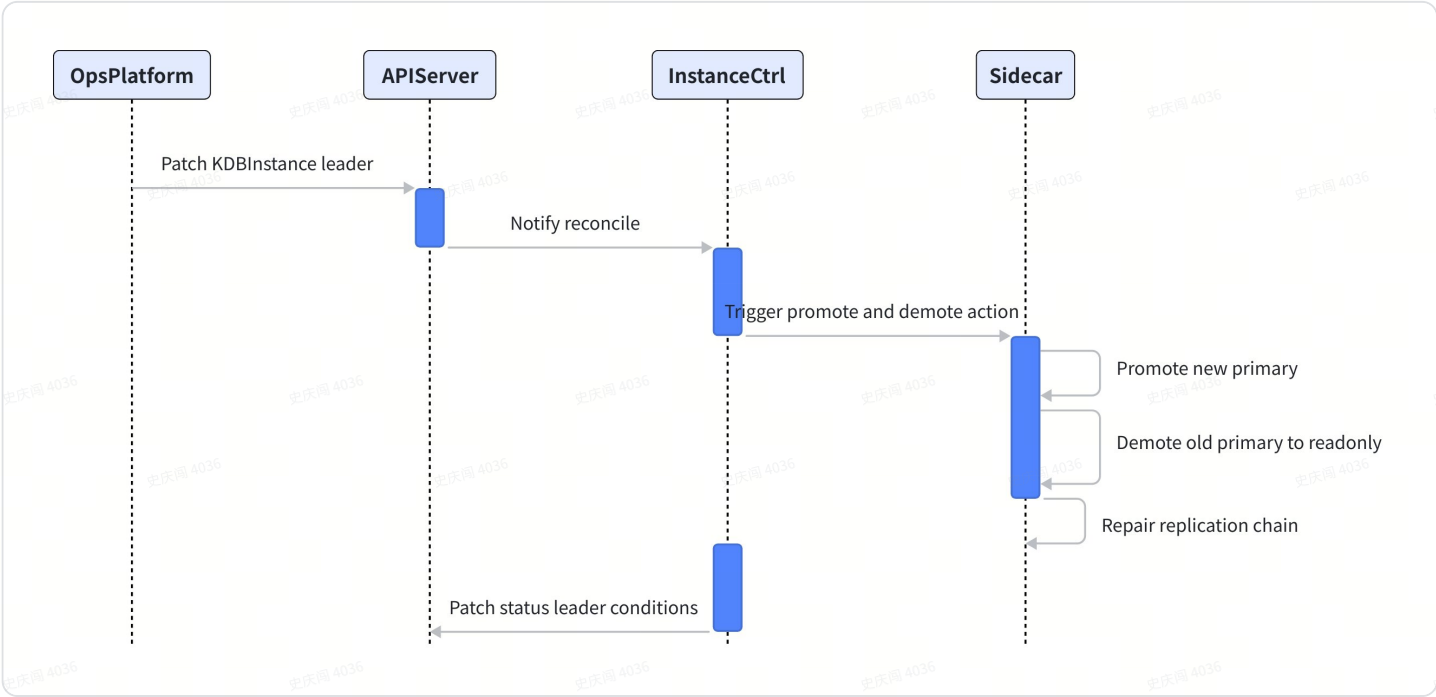




关键交互时序：创建集群（Cluster -> 多 Instance）



关键交互时序：主从切换（控制面决策、数据面执行）



需求拆解

为了降低落地风险，按“先交付声明式基础能力，再逐步补齐高阶能力”的顺序拆解：

--	--	--	--	--

阶段	交付内容	验收要点
MVP	- CRD: <code>KDBCluster</code> / <code>KDBInstance</code> - Instance Controller: 基础资源编排 (SS/PVC/SVC/CM/Secret/RBAC) <code>kdblet</code>: 初始化与健康检查 <code>kdbmonitor</code>: 基础指标导出	- 能创建/删除实例, 状态可收敛 <code>shutdown=true</code> 可缩容到 0 且保留 PVC - Prometheus 可抓取基础指标
v1.0	- Cluster Controller: 按 <code>spec.instances</code> 管理实例集合 (增删) - Step/Task 框架: 统一幂等收敛模型 - 引擎插件: MySQL/PG 的差异化 StepManager	- 集群维度扩缩容 (新增/删除 instance) - Reconcile 幂等、无副作用重试 - 引擎切换不影响通用编排
v1.1 (演进)	- 备份与恢复: CronJob/Job 编排 + <code>kdblet</code> 执行 - 切换: 平台/Operator 决策新主, <code>kdblet</code> 执行 Promote/Demote - 可选接入层: Proxy 模式与路由更新	- 备份可用、可追溯 (对象存储位置与元数据) - 切换后读写入口与状态一致 - 关键动作均产出 Event 与指标



风险与应对:

- 引擎差异导致逻辑膨胀: 通过 StepManager 抽象并约束公共接口。
- 控制器误操作造成数据风险: 删除流程必须由 Finalizer 串行化关键动作; 默认不自动清理 PVC。
- 容器内动作失败难定位: 所有动作必须产出结构化日志与事件, 并暴露关键指标。

详细设计

5.1 CRD 设计

KDBCluster (集群级)

- `spec.engine`：引擎类型，例如 `mysql`、`pg`。
- `spec.deployArch`：部署架构，例如 `Single`、`Master-Slave`、`Master-Replica`（后续可扩展）。
- `spec.instances[]`：实例清单，按名称编排创建/删除。
- `spec.leader`：架构需要时的主库描述（约束由架构与引擎共同决定）。

KDBInstance (实例级)

- `spec.instance`：实例规格（镜像、资源、亲和/反亲和、PVC 规格、监控容器等）。
- `spec.leader`：主从架构下的主库定位信息。
- `spec.shutdown`：逻辑关停（`StatefulSet.spec.replicas=0`，保留 PVC）。

5.2 Controller 设计与幂等策略

- **KDBCluster Controller**：以 `cluster.spec.instances` 为意图源，确保对应的 `KDBInstance` 资源存在/删除；仅做“实例集合编排”，不进入实例细节。
- **KDBInstance Controller**：负责实例级资源编排，基于引擎选择 StepManager 组合不同 Step。

幂等策略：

- 所有 Step 以“意图 (Intent) = 期望状态”为中心，重复执行不会产生副作用。
- 删除流程使用 Finalizer 管理，确保关键清理动作（缩容、解绑、状态回写）在对象最终删除前完成。
- 支持“命名空间暂停收敛”，在 Reconcile 入口进行短路（用于紧急止血/灰度）。

5.3 数据面 (Sidecar) 接口与职责

`kdblet` 提供统一的动作 API（可通过 HTTP/gRPC 或命令协议实现，具体实现独立于 Operator）：

- 初始化：数据目录初始化、权限初始化、（可选）导入基础配置。
- 探活：Readiness/Liveness 检测、主从状态探测。
- 复制搭建/修复：根据引擎实现差异化流程。
- 切换：Promote 新主、旧主降级只读、修复复制链路。
- 备份：触发全量/增量备份并上传对象存储，返回备份集位置与元信息。

 **接口边界**：Operator 只负责“何时触发、对哪个实例触发、状态如何回写”；`kdblet` 负责“怎么做”。

5.4 存储与网络模型

- 存储：
 - 数据盘：`spec.instance.dataVolumeClaimSpec`。
 - 日志盘（可选）：`spec.instance.logVolumeClaimSpec`，用于 WAL/redo 隔离。
 - 关停保留：`shutdown=true` 时缩容到 0，但 PVC 保留用于后续恢复。
- 网络：
 - 对内：`Service` + `Endpoints` 为业务/运维提供统一入口。
 - 对外（可选）：`KdbProxy` 提供读写分离、连接池或稳定连接能力。

5.5 可观测性与安全

- 指标：`kdbmonitor` 暴露 exporter 指标，Prometheus 抓取，Grafana 展示。
- 事件：Operator 在关键动作写入 K8s Event，用于审计与排障。
- 日志：控制面用于定位收敛/资源编排问题；数据面用于定位初始化/复制/备份等动作失败。
- 安全：敏感信息通过 Secret 下发（例如全局配置 `kdb-global-config`），RBAC 最小化按需授权；如 `kdblet` 需要访问 K8s API，采用最小权限 ServiceAccount。